Algorithmic Methods for Mathematical Models

– COURSE PROJECT –

A manufacturing plant operates over a planning horizon of N discrete time periods, indexed by $i \in \{1, 2, \ldots, N\}$. A critical high-precision machine can either be running in production mode (i.e., in ON state) or be shut down for maintenance (i.e., in OFF state) during each time period. Due to operational constraints, the machine must obey strict continuous-run duration rules:

- **Minimum Run Length:** To prevent premature wear from frequent startup and cool-down cycles, once the machine is switched ON, it must remain continuously ON for at least L consecutive time periods before it can be switched OFF.
- **Maximum Run Length:** To prevent overheating, the machine cannot remain continuously ON for more than U consecutive time periods without being turned OFF for at least one period.

As regards boundary conditions, consider that the machine was turned OFF prior to period 1, and should also be OFF right after time N.

Moreover, in each time period i, running the machine requires a varying amount of energy E_i, and yields a profit P_i. The plant has a minimum profit threshold T to be reached, and for economic and sustainability reasons energy consumption should be minimized.

Inputs satisfy that $1 \leq L \leq U \leq N$, and also $E_i, P_i, T \geq 0$. You can assume all input data are integers.

Consider the following input data:

$$N = 6 \qquad T = 14$$
$$L = 2 \qquad P = \begin{pmatrix} 1 & 9 & 1 & 3 & 4 & 5 \end{pmatrix}$$
$$U = 5 \qquad E = \begin{pmatrix} 1 & 2 & 2 & 3 & 3 & 3 \end{pmatrix}$$

A solution with energy consumption 12 consists in having the machine ON from time 1 to time 2, both included, and from time 4 to time 6, again both included. Note that this is allowed, as run lengths are 2 and 3 respectively, and $L = 2$ and $U = 5$. Besides, profit is 22, which is enough for $T = 14$. Another solution has energy consumption 11 and consists in having the machine ON from time 1 to time 5, both included. Note that this is allowed, as $L = 2$ and $U = 5$. Moreover, profit is 18, which is sufficient for $T = 14$. Yet an even better solution consists in having the machine ON from time 1 to time 4, both included. Since $L = 2$ and $U = 5$, this is permitted. Profit is $14 = T$, and energy consumption is 8. In fact, this solution can be proved to be optimal. Also notice that having the machine ON all the time is not possible, since $U = 5$. And always having the machine OFF is not possible either, as then the minimum profit T cannot be reached.

A checker is provided to help you debug your codes. The checker, given a `.dat` file with the input data and a `.sol` file with the solution in the format described below, tests whether all constraints are satisfied. On the other hand, **it is not checked if the solution is optimal** (because that would require solving the optimization problem!). The checker is written in Python and should be called e.g.:

`python checker.py input.dat output.sol`.

Solutions should be written as a sequence of lines. Each line should contain a pair of the form `a␣b` where $1 \leq$ `a` $\leq$ `b` $\leq N$, meaning that the machine is ON from time `a` to time `b`, both included. Pairs should not overlap. For instance, the first of the solutions in the above example could be represented as follows:

```
1 2
4 6
```

1. **Work to be done:**

   (a) Build an integer linear programming model for the problem and implement it in OPL. Your code should include an execute block for writing the optimal solution in the format described above. Simplicity and efficiency will be taken into account when grading.

   (b) Because of the complexity of the problem, heuristic algorithms can also be applied. Here we will consider the following:

      i. a greedy constructive algorithm,

      ii. a greedy constructive + a local search procedure,

      iii. GRASP as a meta-heuristic algorithm. You can reuse the local search procedure that you developed in the previous step.

      Design the three algorithms and implement them in the programming language you prefer. Your programs should write the final solution in the format described above.

   (c) *Tuning of parameters and instance generation.* Given a problem instance, we define its *size* as the value of N.

      i. Implement an instance generator that produces random instances for a given size.

      ii. Tune the α parameter of the GRASP constructive phase with a set of randomly generated instances of large enough size.

      iii. Generate problem instances with increasingly larger size. Solving each instance with CPLEX should take from 1 to 30 min.

   (d) Compare the performance of CPLEX with the meta-heuristic algorithms, both in terms of computation time and of quality of the solutions as a function of the size of the instances.

   (e) Prepare a report and a presentation of your work on the project.

2. **Report:**

   Prepare a report (8-10 pages) in PDF format including:

   - The formal problem statement.
   - The integer linear programming model, with a definition and a short description of the variables, the objective function and the constraints. Do not include OPL code in the document, but rather their mathematical formulation.
   - For the meta-heuristics, the pseudo-code of your constructive, local search, and GRASP algorithms, including equations for describing the greedy cost function(s) and the RCL.
   - Tables or graphs with the tuning of parameters, and with the comparative results.

   Together with the report, you should also give all sources (OPL code, programs of the meta-heuristics, instances **and their corresponding solutions**, instance generator) and instructions on how to use them, so that results can be reproduced.

3. **Presentation:**

   You are expected to make a presentation of your work at the end of the course (at most 10 minutes long; overtime presentations will be terminated). All group members must participate in the presentation and know all the work in the project. The slots of $\{10, 14, 17\}/12/26$ will be devoted to these presentations. The schedule will be announced in due time.

   The slides of the presentation as well as the report should be delivered in PDF format by **08/12/26**.

   The slides should include the full model, the pseudocode of the algorithms, as well as any other important mathematical equation. As for the results, we expect you to present graphs comparing the performance of CPLEX on the ILP model and of your meta-heuristics in terms of the quality of the solutions and solving time. Problem instance generation should be also described.

   Please, minimize the auxiliary text in the slides to just what is strictly needed to understand the main content (model, algorithms and results). You are expected to give a full explanation of those contents in the presentation. On the other hand, the report should also contain that explanation in a well-organized way as a text.

Some final remarks:

- Please use `checker.py` to check all your solutions, those generated with CPLEX and also those obtained with your meta-heuristics, and report you did so in the report as well as in the presentation.
- **The use of LLMs (ChatGPT, Claude, Gemini, etc.) for solving the project is forbidden.**